

SentraAuth-CLI: Hardened Authentication System with a Role-Based Security Finding Tracker Case Study

Written to fulfill the task of
Cybersecurity Review



Author:

Aldi Syarif Alhakim	2521010106
Alezuna M. Nadhif Pohan	2521010108

Class: 3CS1

**CEP-CCIT FACULTY OF ENGINEERING
UNIVERSITY OF INDONESIA 2026**

PROJECT ON
SentraAuth-CLI

Developed by

Aldi Syarif Alhakim

Alezuna M. Nadhif Pohan

SentraAuth-CLI

Batch Code : 3CS1

Start Date : 9 September 2026

End Date : 17 September 2026

Name of Faculty : Tri Agus Riyadi, S.Kom., M.T.

Names of Developer :

Aldi Syarif Alhakim

Alezuna M. Nadhif Pohan

Date of Submission: 17 September 2026

ACKNOWLEDGEMENT

The author expresses gratitude for the completion of this final project report entitled "SentraAuth-CLI". This project explores the application of modular, function-based design in the Python language, specifically in the design and implementation of a hardened authentication and access control system based on a Command Line Interface (CLI) that is secure, maintainable, and easy to extend further.

Through the application of design principles such as separation of concerns, data-driven access control, and defense-in-depth, the system ensures clear module boundaries, reusable authorization logic, and consistent security enforcement across features. This report also highlights the integration between password hashing using bcrypt, multi-factor authentication based on the TOTP standard, persistent data storage in a MySQL database, and centralized audit logging within a terminal-based environment.

The author acknowledges that this report still has limitations, including the plaintext storage of the MFA secret and the absence of session expiry, both of which are discussed openly in this report. Therefore, constructive criticism and suggestions are greatly appreciated for improvement in future work. It is hoped that this report will be beneficial to readers.

Depok, September 17th, 2026
Author

SYSTEM ANALYSIS

This report examines the design and development of a hardened CLI authentication system, SentraAuth-CLI, that runs directly in the terminal, focusing on reliability, data security, and role-based access control. The application provides a lightweight authentication and access control layer without requiring a graphical interface, built as a direct response to the security weaknesses commonly found in introductory database login scripts.

The system addresses vulnerability classes such as plaintext password storage, absence of brute-force protection, and hardcoded authorization logic by providing password hashing, account logout, data-driven role-based access control, multi-factor authentication, and audit logging. Its scope includes user authentication, account management, a self security-audit module, and a role-based Security Finding Tracker case study. Network-layer protections such as firewalls and TLS, as well as graphical interfaces, are excluded from the current scope.

The application was developed in Python using mysql-connector-python for database access, bcrypt for password hashing, pyotp and qrcode for TOTP-based multi-factor authentication, rich for terminal styling, and python-dotenv for configuration management. A modular architecture separates each responsibility into its own file, authentication, authorization, account management, audit logging, MFA, and the finding tracker, while database operations consistently use parameterized queries to protect against SQL injection.

The system provides menu-driven access covering authentication, account management, password self-service, MFA enrollment, security finding tracking, and self security-auditing. An audit logging mechanism records every authentication and account-related event automatically based on user action. Testing covered login and logout behavior, MFA enrollment and verification, role-scoped access to menus and data, finding submission and search, and self-audit accuracy. Overall, the project demonstrates the application of modular software design, database security practices, and defense-in-depth principles in a complete terminal-based authentication system.

ENTITY RELATIONSHIP DIAGRAM

Entity Relationship Diagram (ERD)

The system uses one relational database named `sentra_auth`, consisting of five core tables. The `roles` table stores the available roles, and the `role_permissions` table maps each role to one or more named permissions, which is the mechanism that makes the access control model data driven rather than hardcoded. The `users` table stores account credentials, lockout state, and MFA status. The `audit_log` table records every authentication and account related event by username. The `findings` table stores the data submitted through the Security Finding Tracker case study module, linked to the reporting user.

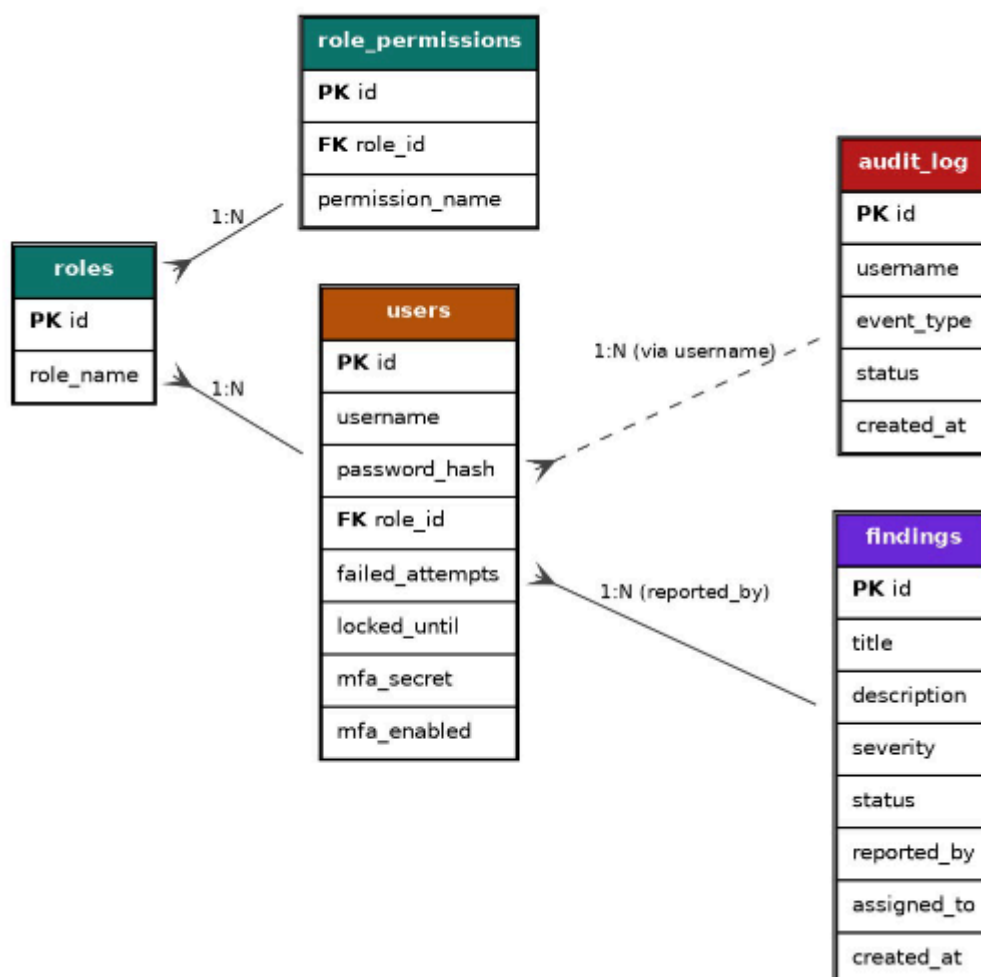


Figure : Entity Relationship Diagram of the `sentra_auth` database

Two relationships are worth noting specifically. First, the relationship between users and

audit_log is maintained through the username column rather than a strict foreign key constraint, since audit records must remain even if the referenced account is later deleted, preserving the historical trail. Second, the relationship between roles and role_permissions is the foundation of the entire authorization model described in the main paper, since every permission check performed throughout the application ultimately queries this pair of tables.

TABLE DESIGN

Table Design

This section describes the structure of each table in detail, including data type, constraint, and the purpose of every column.

2.1 Table: roles

Field Name	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for the role
role_name	VARCHAR(20)	UNIQUE, NOT NULL	Name of the role, for example admin or analyst

2.2 Table: role_permissions

Field Name	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for the mapping row
role_id	INT	FK to roles.id	The role that owns this permission
permission_name	VARCHAR(50)	NOT NULL	Name of the permission, for example manage_accounts

2.3 Table: users

Field Name	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for the account
username	VARCHAR(50)	UNIQUE, NOT NULL	Login identifier chosen by the user
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hash of the password, never stored as plaintext
role_id	INT	FK to roles.id	The role assigned to this account
failed_attempts	INT	DEFAULT 0	Counter of consecutive failed login attempts
locked_until	DATETIME	NULLABLE	Timestamp until which the account remains locked
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time the account was created
mfa_secret	VARCHAR(32)	NULLABLE	TOTP secret key used for MFA verification
mfa_enabled	TINYINT(1)	DEFAULT 0	Flag indicating whether MFA is active for this account

2.4 Table: audit log

Field Name	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for the log entry
username	VARCHAR(50)	NULLABLE	The user associated with the event
event_type	VARCHAR(30)	NOT NULL	Type of event, for example login or account_created
status	VARCHAR(20)	NOT NULL	Outcome of the event, for example success, failed, or locked

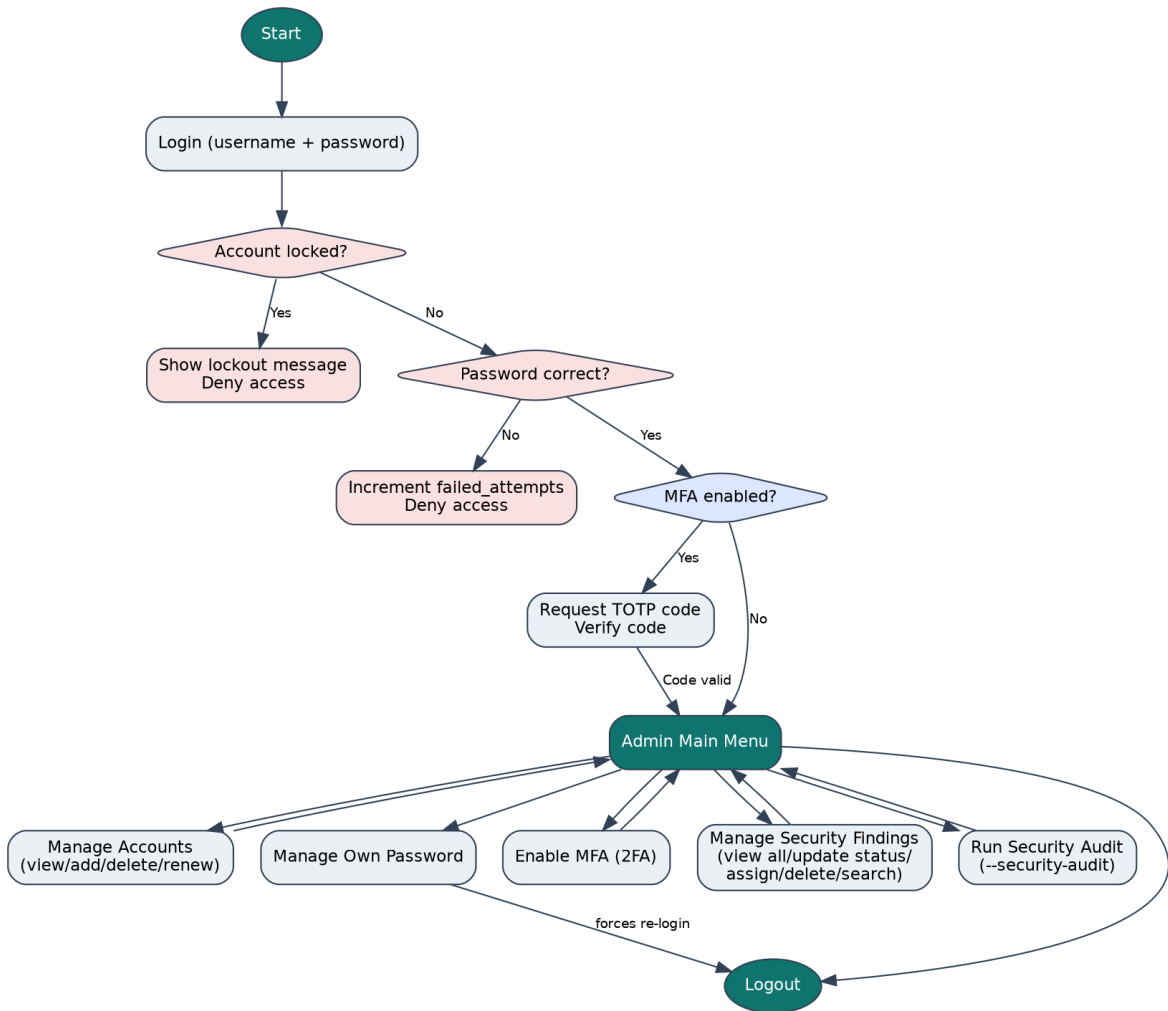
ip_or_host	VARCHAR(100)	DEFAULT localhost-cli	Origin of the request
created_at	DATETIME	DEFAULT CURRENT_TIMES TA MP	Time the event occurred

2.5 Table: findings

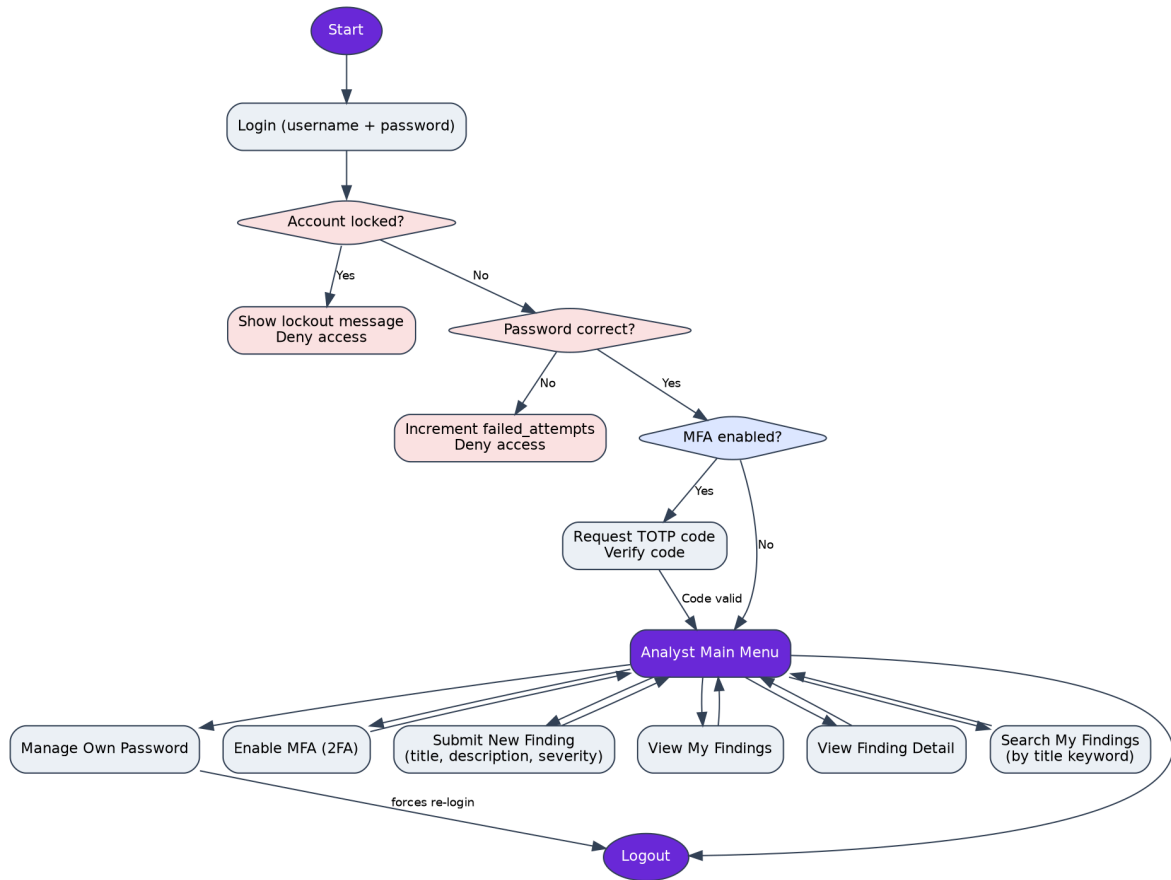
Field Name	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for the finding
title	VARCHAR(150)	NOT NULL	Short title of the security finding
description	TEXT	NULLABLE	Full description of the finding
severity	ENUM	Critical, High, Medium, Low	Severity level of the finding
status	ENUM	Open, In Progress, Resolved	Current resolution status
reported_by	VARCHAR(50)	NOT NULL	Username of the analyst who submitted the finding
assigned_to	VARCHAR(50)	NULLABLE	Username of the person assigned to resolve it
created_at	DATETIME	DEFAULT CURRENT_TIMES TA MP	Time the finding was submitted
updated_at	DATETIME	ON UPDATE CURRENT_TIMES TA MP	Time the finding was last modified

FLOWCHART

Flowchart Admin



Flowchart Analyst



Source Code

4. Source Code (Excerpts)

The following excerpts represent the core logic of each module in the system. For brevity, only the most representative function from each file is shown rather than the complete file.

4.1 db.py

Handles the connection to the MySQL database using credentials loaded from the environment file.

```
def get_connection():
    try:
        conn = mysql.connector.connect(
            host=os.getenv("DB_HOST"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            database=os.getenv("DB_NAME")
        )
        return conn
    except Error as err:
        print(f"[ERROR] Gagal konek ke database: {err}")
        return None
```

4.2 auth.py

Handles password hashing, verification, and the account lockout mechanism.

```
def hash_password(plain_password: str) -> str:
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(plain_password.encode("utf-8"), salt)
    return hashed.decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode("utf-8"),
        hashed_password.encode("utf-8"))

def is_account_locked(locked_until) -> bool:
    if locked_until is None:
        return False
    return datetime.now() < locked_until
```

4.3 rbac.py

Retrieves and checks permissions assigned to a role, forming the basis of the data driven access control model.

```
def get_permissions(role_name: str) -> list:
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    query = ("SELECT role_permissions.permission_name "
        "FROM role_permissions ")
```

```
"JOIN roles ON role_permissions.role_id = roles.id " "WHERE roles.role_name = %s")
cursor.execute(query, (role_name,))
hasil = cursor.fetchall()
return [row["permission_name"] for row in hasil]
```

```
def has_permission(user_permissions: list, required_permission: str) -> bool: return required_permission in
user_permissions
```

4.4 accounts.py

Provides CRUD operations for user accounts, restricted to the admin role.

```
def tambah_akun(actor_username: str):
    username = input("Username baru: ").strip()
    password = input("Password baru: ").strip()
    role_input = input("Role (admin/analyst: ").strip().lower()

    if role_input not in ["admin", "analyst"]:
        print_error("Role tidak valid! Harus admin atau analyst.") return
    if len(password) < 8:
        print_error("Password minimal 8 karakter.")
    return

    pw_hash = hash_password(password)
    cursor.execute(
        "INSERT INTO users (username, password_hash, role_id) VALUES (%s, %s, %s)", (username, pw_hash, role_id)
    )
```

4.5 audit.py

Centralized logging function used by every module to record authentication and account related events.

```
def log_event(username: str, event_type: str, status: str, ip_or_host: str = "localhost-cli"):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO audit_log (username, event_type, status, ip_or_host) VALUES (%s, %s, %s, %s)",
            (username, event_type, status, ip_or_host)
        )
        conn.commit()
    except Exception as err:
        print(f"[WARNING] Audit log gagal dicatat: {err}")
```

4.6 mfa.py

Implements TOTP based multi factor authentication, including secret generation, QR code display, and code verification.

```
def generate_secret() -> str:
    return pyotp.random_base32()

def verifikasi_kode(secret: str, kode_input: str) -> bool:
    totp = pyotp.TOTP(secret)
    return totp.verify(kode_input, valid_window=1)

def tampilkan_qr_terminal(secret: str, username: str):
```

```

totp = pyotp.TOTP(secret)
uri = totp.provisioning_uri(name=username, issuer_name=APP_NAME) qr = qrcode.QRCode()
qr.add_data(uri)
qr.make()
qr.print_ascii(invert=True)

```

4.7 findings.py

Implements the Security Finding Tracker case study module, including role scoped search.

```

def submit_finding(username: str):
    title = input("Judul temuan: ").strip()
    description = input("Deskripsi: ").strip()
    severity = mapping_severity[input("Pilih severity: ").strip()]

    cursor.execute(
        "INSERT INTO findings (title, description, severity, reported_by) VALUES (%s, %s, %s, %s)",
        (title, description, severity, username)
    )
    conn.commit()
    log_event(username, "finding_submitted", "success")

```

4.8 ui.py

Centralizes terminal styling using the rich library, so every module produces a consistent visual output.

```

def print_success(pesan: str):
    console.print(f"[OK] {pesan}", style="bold green")

def print_error(pesan: str):
    console.print(f"[X] {pesan}", style="bold red")

def judul_bagian(teks: str):
    console.print(Panel(teks, style="bold magenta", expand=False))

```

4.9 security_check.py

Performs a self audit of the system, formatted to resemble a conventional VAPT report.

```

def cek_kredensial_default() -> list:
    temuan = []
    for admin in get_admin_accounts():
        if verify_password(DEFAULT_ADMIN_PASSWORD, admin["password_hash"]): temuan.append({
            "finding": "Kredensial Default Masih Aktif",
            "severity": "HIGH",
            "status": "FAIL",
            "rekomendasi": "Segera ganti password akun ini." })
    return temuan

```

4.10 main.py

Serves as the entry point of the application, managing the login flow and rendering the menu according to the permissions held by the logged in user.

```

def menu_utama():
    menu_tersedia = []
    if has_permission(current_session["permissions"], "manage_accounts"):

```

```
menu_tersedia.append(("kelola_akun", "Kelola Akun Sistem")) if  
has_permission(current_session["permissions"], "submit_finding"):  
menu_tersedia.append(("findings_user", "Temuan Saya"))
```

```
tabel_menu(menu_tersedia)  
pilihan = input("Pilih menu: ").strip()
```

4.11 seed_users.py

A one time setup script used to create the default admin and analyst accounts with properly hashed passwords.

```
def seed():  
    data_default = [  
        ("admin", hash_password("adminjuga"), 1),  
        ("budi", hash_password("budi123"), 2),  
    ]  
    for username, pw_hash, role_id in data_default:  
        cursor.execute(  
            "INSERT INTO users (username, password_hash, role_id) VALUES (%s, %s, %s)",  
            (username, pw_hash, role_id)  
        )  
    conn.commit()
```

SYSTEM FLOW: ADMIN AND ANALYST ROLE

SentraAuth-CLI enforces two distinct user roles through its data-driven RBAC (Role-Based Access Control) engine: Admin and Analyst. Each role is granted a different set of menu options at runtime by the `rbac.py` module, based on permissions stored in the `role_permissions` table rather than hardcoded conditions in the source code.

This section documents the actual menu flow experienced by each role, captured directly from the running application, and closes with a permission matrix summarizing what each role can and cannot do.

Role Permission Matrix

Capability	Admin	Analyst
Login & MFA verification	Yes	Yes
Change own password	Yes	Yes
Enable / manage own MFA (2FA)	Yes	Yes
View all system accounts	Yes	No
Add / delete accounts	Yes	No
Renew another account's password	Yes	No
View system-wide audit log	Yes	No
Run self security-check audit	Yes	No
Submit a new security finding	No	Yes
View own submitted findings only	No	Yes
View all findings from every user	Yes	No
Update a finding's status	Yes	No
Assign a finding to a user	Yes	No
Delete a finding	Yes	No

ADMIN ROLE: SYSTEM FLOW

The Admin role has full administrative control over the system: it can manage every user account, review the system-wide audit trail, run the self security-check module, and manage every security finding submitted by any user. The walkthrough below follows an actual admin session from login to each available menu.

Login and Two-Factor Authentication (MFA) Verification

1. From the main SentraAuth-CLI menu, the admin selects option "1. Login" and is presented with the login form.
2. The admin enters their username and password. Credentials are verified against the bcrypt hash stored in the users table (see `auth.py, verify_password()`).
3. Because this account has MFA enabled, the system requests a 6-digit code from the admin's authenticator app before granting access (`mfa.py, verifikasi_kode()`).
4. On success, the system prints a welcome message that includes the authenticated role ("Role: ADMIN") and immediately renders the Admin Main Menu, which exposes five options: Manage System Accounts, Change My Password, Enable MFA, and Manage Security Findings, in addition to Logout.

```
SENTRAAUTH CLI
1. Login
2. Keluar Aplikasi
Pilih: 1

FORM LOGIN

Username: admin
Password: admin12345

[MFA] Akun ini dilindungi Two-Factor Authentication.
Masukkan kode 6 digit dari app authenticator: 728356
[✓] Selamat datang, admin (Role: ADMIN)

MENU UTAMA (ADMIN)

Login sebagai: admin
0. Logout
1. Kelola Akun Sistem
2. Ganti Password Saya
3. Aktifkan MFA (2FA)
4. Kelola Temuan Keamanan
Pilih menu: 
```

Figure : Admin login flow, MFA code verification, and the resulting Admin Main Menu.

Selecting option "1. Kelola Akun Sistem" from the main menu opens the account management submenu, which is only rendered because the admin's role carries the manage_accounts permission.

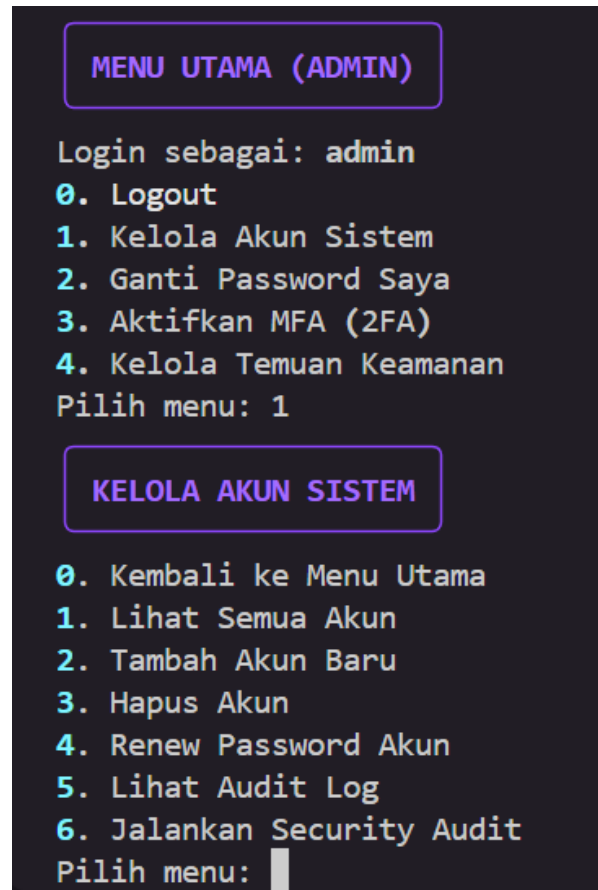


Figure : Admin Main Menu → Manage System Accounts submenu.

The submenu exposes six actions: view all accounts, add a new account, delete an account, renew an account's password, view the audit log, and run a security audit. Selecting "1. Lihat Semua Akun" lists every account in the system together with its assigned role, as shown below.

MENU UTAMA (ADMIN)

Login sebagai: admin

0. Logout

1. Kelola Akun Sistem

2. Ganti Password Saya

3. Aktifkan MFA (2FA)

4. Kelola Temuan Keamanan

Pilih menu: 1

KELOLA AKUN SISTEM

0. Kembali ke Menu Utama

1. Lihat Semua Akun

2. Tambah Akun Baru

3. Hapus Akun

4. Renew Password Akun

5. Lihat Audit Log

6. Jalankan Security Audit

Pilih menu: 1

Daftar Akun Sistem

No	Username	Role
1	admin	ADMIN
2	budi	ANALYST

Figure : Full account listing, showing both the admin and analyst accounts and their roles.

The remaining options in this submenu (Tambah Akun Baru, Hapus Akun, and Renew Password Akun) follow the same pattern: the admin is prompted for the required fields (username, temporary password, target account) and the action is written directly to the users table, with every action recorded to the audit log described in Section A.3.

View Audit Log

Option "5. Lihat Audit Log" prints the centralized audit trail maintained by audit.py. Every authentication event, account change, and finding-management action performed by any user is timestamped and recorded here, regardless of whether it succeeded or failed.

Audit Log			
Waktu	Username	Event	Status
2026-09-17 13:58:57	admin	self_password_change	SUCCESS
2026-09-17 13:54:23	admin	login	SUCCESS
2026-09-17 11:28:31	admin	login	SUCCESS
2026-09-17 04:46:51	admin	login	SUCCESS
2026-09-17 04:44:06	budi	login	SUCCESS
2026-09-17 04:41:48	admin	finding_status_updated	SUCCESS
2026-09-17 04:41:17	admin	finding_assigned	SUCCESS
2026-09-17 04:39:39	admin	account_deleted	SUCCESS
2026-09-17 04:38:05	admin	login	SUCCESS
2026-09-17 04:32:46	budi	finding_submitted	SUCCESS
2026-09-17 04:31:40	budi	finding_submitted	SUCCESS
2026-09-17 04:31:06	budi	login	SUCCESS
2026-09-17 04:29:25	admin	login	SUCCESS
2026-09-17 04:24:05	admin	login	SUCCESS
2026-09-17 04:18:38	admin	login	SUCCESS
2026-09-17 04:16:06	admin	login	SUCCESS
2026-09-16 20:32:08	admin	login	SUCCESS
2026-09-16 16:46:10	budi	finding_submitted	SUCCESS
2026-09-16 16:45:54	budi	login	SUCCESS
2026-09-16 16:45:23	admin	login	SUCCESS

Figure : Audit log showing login events, an account deletion, and finding-related activity.

Run Self Security-Check Audit

Option "6. Jalankan Security Audit" triggers the self security-check module, which inspects the system's own configuration (default credentials, MFA coverage on admin accounts, .env exclusion from version control, and the recent lockout pattern) and reports the result in a VAPT-style table with a severity, a pass/fail status, and a remediation recommendation where relevant.

Hasil Security Audit				
Finding	Severity	Status	Detail	Rekomendasi
Kredensial Default Masih Aktif	HIGH	PASS	Tidak ada akun admin yang menggunakan password default.	-
Admin Tanpa MFA	MEDIUM	PASS	Semua akun admin sudah mengaktifkan MFA.	-
File .env Tidak Ter-exclude dari Version Control	HIGH	PASS	File .env sudah ter-exclude dengan benar di .gitignore.	-
Pola Brute-Force 24 Jam Terakhir	INFO	INFO	Tercatat 0 kejadian lockout akun dalam 24 jam terakhir.	Tidak ada tindakan diperlukan.

Figure : Self security-check audit results.

Option "4. Kelola Temuan Keamanan" on the main menu opens the finding-management submenu, which grants the admin visibility and control over every finding submitted by any analyst.

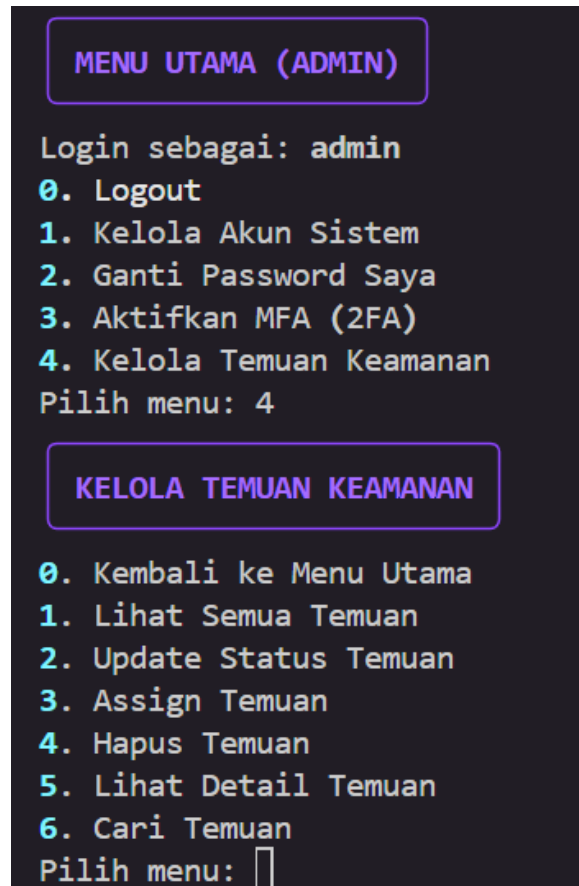


Figure : Admin Main Menu → Manage Security Findings submenu.

Selecting "1. Lihat Semua Temuan" lists every finding in the system across all reporters, together with its severity, status, and current assignment.

KELOLA TEMUAN KEAMANAN

0. Kembali ke Menu Utama
1. Lihat Semua Temuan
2. Update Status Temuan
3. Assign Temuan
4. Hapus Temuan
5. Lihat Detail Temuan
6. Cari Temuan
Pilih menu: 1

Semua Temuan Sistem

ID	Judul	Severity	Status	Reporter	Assigned To
2	SQLInjection	High	Open	budi	-
1	JailBreaking	High	In Progress	budi	Budi

Figure : Full findings list, visible to the admin across all reporters.

Selecting "5. Lihat Detail Temuan" and entering a finding ID shows the complete record for that finding, including its description, reporter, current assignee, status, and creation timestamp.

KELOLA TEMUAN KEAMANAN

0. Kembali ke Menu Utama
1. Lihat Semua Temuan
2. Update Status Temuan
3. Assign Temuan
4. Hapus Temuan
5. Lihat Detail Temuan
6. Cari Temuan
Pilih menu: 5

DETAIL TEMUAN

Masukkan ID temuan: 1

ID: #1
Judul: JailBreaking
Severity: High
Status: In Progress
Dilaporkan oleh: budi
Ditugaskan ke: Budi
Deskripsi:
INPUTNYA
Dibuat: 2026-09-17 04:31:40

Figure : Detail view of a single finding.

The remaining submenu options (Update Status Temuan, Assign Temuan, Hapus Temuan, and Cari Temuan) let the admin move a finding through its resolution lifecycle (Open, In Progress, Resolved), assign it to a specific user, remove invalid or duplicate entries, and search the finding list by keyword.

Change My Password

Option "2. Ganti Password Saya" on the main menu lets the admin change their own password. The system requires the current password before accepting a new one, hashes the new password with bcrypt, and prompts the admin to log in again with the updated credentials.

```
MENU UTAMA (ADMIN)
Login sebagai: admin
0. Logout
1. Kelola Akun Sistem
2. Ganti Password Saya
3. Aktifkan MFA (2FA)
4. Kelola Temuan Keamanan
Pilih menu: 2

=== GANTI PASSWORD SAYA ===
Masukkan password lama: admin12345
Masukkan password baru: admin123456
Konfirmasi password baru: admin123456
Berhasil! Password Anda telah diperbarui. Silakan login ulang.
```

Figure : Self-service password change flow.

Enable MFA (Two-Factor Authentication)

Option "3. Aktifkan MFA (2FA)" starts the enrollment flow. The sequence below follows a freshly created admin account, "aldi", that has not yet enabled MFA: the login step completes with only a username and password, and the main menu is reached directly.

```
SENTRAAUTH CLI

1. Login
2. Keluar Aplikasi
Pilih: 1

FORM LOGIN

Username: aldi
Password: aldi12345
[✓] Selamat datang, aldi (Role: ADMIN)

MENU UTAMA (ADMIN)

Login sebagai: aldi
0. Logout
1. Kelola Akun Sistem
2. Ganti Password Saya
3. Aktifkan MFA (2FA)
4. Kelola Temuan Keamanan
Pilih menu: 3
```

Figure : Login for an account without MFA enabled yet, followed by selecting "3. Aktifkan MFA".

The system generates a local secret and displays it as a scannable QR code, which the admin scans with an authenticator app such as Google Authenticator or Authy. The same secret is also shown as text for manual entry.



Figure : QR code and manual secret key shown during MFA enrollment.

The admin then enters the 6-digit code currently displayed in the authenticator app to confirm the pairing. Only once this code is verified does the system persist the secret and mark MFA as enabled for the account, after which the admin is returned to the main menu.

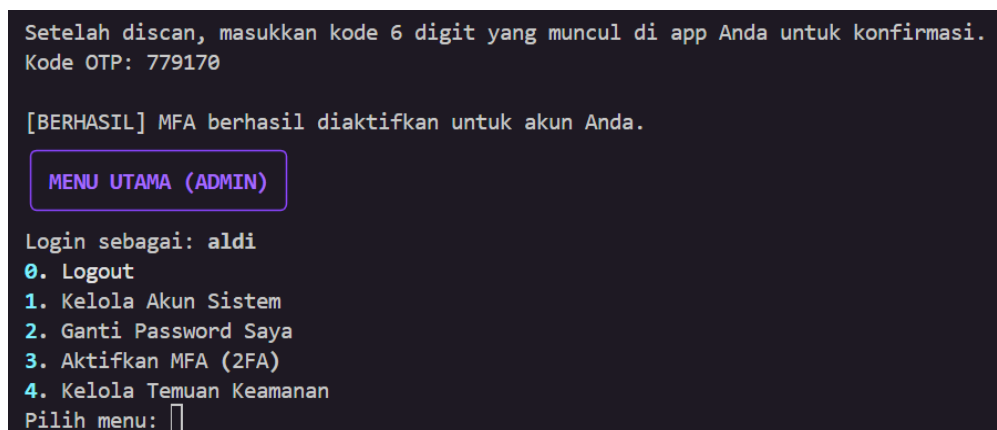


Figure : OTP confirmation completing MFA enrollment.

ANALYST ROLE: SYSTEM FLOW

The Analyst role is scoped to reporting and tracking security findings. Unlike the admin, an analyst cannot manage other accounts, view the system-wide audit log, or run a security audit; the main menu rendered after login reflects this narrower permission set.

Login and Two-Factor Authentication (MFA) Verification

The login mechanism itself is identical to the admin flow described in Section A.1, since both roles share the same `auth.py` and `mfa.py` modules. Logging in as the analyst account "budi", which already has MFA enabled, requires the correct password followed by a valid 6-digit TOTP code.

On success, the Analyst Main Menu is rendered with only three options: Change My Password, Enable MFA, and Temuan Saya (My Findings), in addition to Logout. Note the absence of Manage System Accounts and Manage Security Findings, which are Admin-only.

```
FORM LOGIN
Username: budi
Password: budi12345

[MFA] Akun ini dilindungi Two-Factor Authentication.
Masukkan kode 6 digit dari app authenticator: 408116
[✓] Selamat datang, budi (Role: ANALYST)

MENU UTAMA (ANALYST)

Login sebagai: budi
0. Logout
1. Ganti Password Saya
2. Aktifkan MFA (2FA)
3. Temuan Saya
Pilih menu: █
```

Figure : Analyst login flow and the resulting Analyst Main Menu, showing a reduced option set compared to Figure A.1.

My Findings (Temuan Saya)

Option "3. Temuan Saya" opens the analyst's personal findings submenu, which mirrors the shape of the admin's finding-management menu but is scoped entirely to findings the analyst reported themselves.

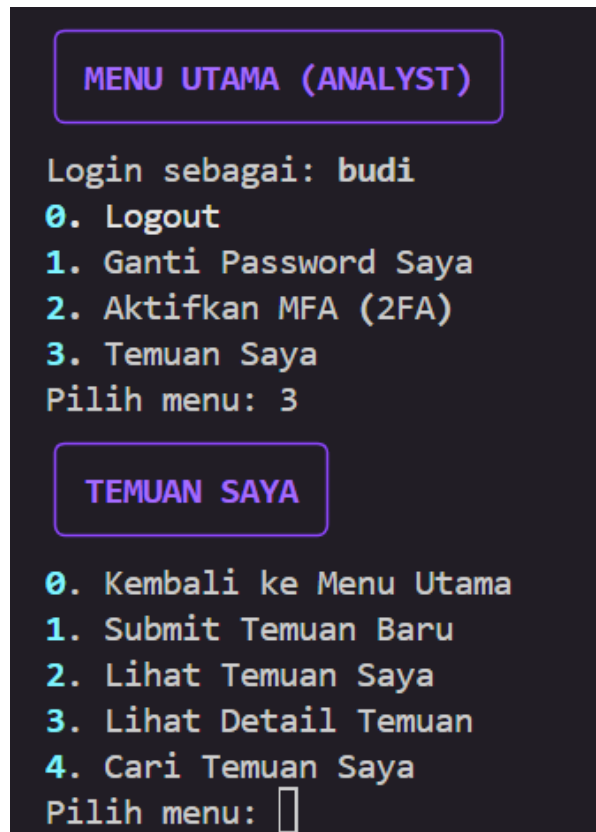


Figure : Analyst Main Menu → My Findings submenu.

Selecting "1. Submit Temuan Baru" prompts the analyst for a title, description, and severity level, and submits a new finding; the reporting analyst's identity is attached automatically. Selecting "2. Lihat Temuan Saya" lists only the findings that analyst has personally submitted, as shown below, in contrast to the admin's system-wide view in Figure A.7.

TEMUAN SAYA 0. Kembali ke Menu Utama 1. Submit Temuan Baru 2. Lihat Temuan Saya 3. Lihat Detail Temuan 4. Cari Temuan Saya Pilih menu: 2				
Temuan Saya				
ID	Judul	Severity	Status	Assigned To
2	SQLInjection	High	Open	-
1	JailBreaking	High	In Progress	Budi

Figure : An analyst's own findings list, scoped to their own submissions only.

"3. Lihat Detail Temuan" and "4. Cari Temuan Saya" behave the same way as their admin-menu counterparts described in Section A.5, but are restricted by `rbac.py` to only return records where the analyst is the original reporter.

Change My Password and Enable MFA

Options "1. Ganti Password Saya" and "2. Aktifkan MFA (2FA)" on the Analyst Main Menu use the exact same underlying flow documented for the admin in Sections A.6 and A.7 respectively; the account-security features are identical for both roles and are not duplicated here.

REQUIREMENT

Device:

- Laptop Thinkpad X1 Carbon

Software:

1. Kiro
2. MySQL 8.4 Configurator
3. Google Docs
4. Canva

Link : https://drive.google.com/drive/folders/1rEqPJqp9kTS-Zv7tDq7OtGe7D40g-_2v?usp=drive_link

No	Extension	Fungsi
1	bcrypt==5.0.0	Hashing & verifikasi password
2	mysql-connector-python==26.7.0	Koneksi ke database MySQL
3	PyOTP==2.10.0	Generate & verifikasi kode TOTP (MFA)
4	qrcode==8.2	Generate QR code untuk setup MFA
5	python-dotenv==1.2.3	Load kredensial dari file .env
6	rich==15.0.0	Render tabel & panel di tampilan CLI